

Práctica Azure OpenAI

Parte 1 — Setup · Playground · API · Chatbot · Parámetros

★ Esta práctica replica paso a paso los ejemplos vistos en clase. El código base se proporciona completo.

Secciones 1–6 · Puntuación base: 6 puntos



1

Setup

Recurso Azure · modelo · credenciales · entorno Python

2

Chat Playground

Interfaz visual · parámetros · pruebas sin código

3

Llamada Simple

Primera llamada a la API

4

Chatbot con historial

Conversación con memoria y system prompt propio

5

Parámetros y Prompts

temperature · 3 técnicas de prompting (referencia)

6

Asistente configurable

Proyecto integrador — único entregable de la parte base

1

Setup

Recurso Azure · Desplegar modelo · Credenciales · Entorno Python

1.1 Crear el recurso Azure OpenAI

Entra en **portal.azure.com** con tu cuenta y sigue estos pasos en orden:

1	En el buscador superior escribe Azure OpenAI y selecciónalo.
2	Haz clic en + Create (esquina superior izquierda).
3	Subscription: selecciona tu suscripción de estudiante.
4	Resource group → Create new: ponle nombre, ej. rg-openai-practica .
5	Region: selecciona Sweden Central — mejor disponibilidad de cuota.
6	Name: nombre único global, ej. openai-tuapellido .
7	Pricing tier: Standard S0.
8	Pulsa Review + Create → Create . Tardará 1-2 minutos.
9	Cuando aparezca 'Deployment complete', pulsa Go to resource .

■ Si ves 'InsufficientQuota' al crear el recurso, tu suscripción ya tiene un recurso Azure OpenAI activo. Ve a [portal.azure.com](#) → Subscriptions → tu suscripción → Resources para localizarlo. Si lo borraste recientemente, espera 48 horas a que se libere la cuota.

1.2 Desplegar el modelo GPT-4o

Crear el recurso no es suficiente: debes desplegar un modelo dentro de él. El recurso es el contenedor; el deployment es el modelo que usarás desde el código.

1	Desde el recurso, haz clic en Go to Azure AI Foundry portal .
2	En el menú lateral selecciona Model deployments .
3	Haz clic en + Deploy model → Deploy base model .
4	Busca y selecciona gpt-4o . Sin cuota: prueba gpt-4o-mini o gpt-35-turbo (0125) .
5	Deployment name: pon gpt4o-practica (o el que quieras). Apúntalo.

6	Deployment type: prueba Standard. Si da error de cuota → Global Standard.
7	Deja el resto por defecto y pulsa Deploy .

★ El Deployment name es distinto al nombre del modelo. Si despliegas gpt-4o con el nombre gpt4o-practica, en el código usarás gpt4o-practica, NO gpt-4o. Este es el error más frecuente.

1.3 Obtener la API Key y el Endpoint

Vuelve a tu recurso en portal.azure.com y en el menú lateral busca **Resource Management** → **Keys and Endpoint**.

- Copia la **KEY 1** usando el icono de copiar. No la escribas a mano.
- Copia el **Endpoint**. Formato: **https://nombre-recurso.openai.azure.com/**
- Apunta también el **Deployment name** que elegiste en el paso 1.2.

■ Nunca compartas tu API Key ni la subas a GitHub. Por eso se guarda en un archivo .env excluido del repositorio con .gitignore.

1.4 Preparar el entorno en VSCode

Crea una carpeta para la práctica y ábrela en VSCode (File → Open Folder). Abre el terminal integrado (Terminal → New Terminal) y ejecuta:

```
pip install openai python-dotenv tiktoken faiss-cpu numpy
```

Estructura de archivos al final de la práctica:

```
mi-practica/
■■■■ .env # credenciales (NUNCA subir a git)
■■■■ .gitignore
■■■■ llamada_simple.py
■■■■ chatbot.py
■■■■ asistente.py
```

1.5 Crear el archivo .env

En la raíz del proyecto crea el archivo **.env** (con el punto) y pega tus credenciales:

```
AZURE_OPENAI_KEY=pega-aqui-tu-KEY1-del-portal
AZURE_OPENAI_ENDPOINT=https://nombre-de-tu-recurso.openai.azure.com/
AZURE_OPENAI_DEPLOYMENT=gpt4o-practica
AZURE_OPENAI_API_VERSION=2024-02-01
```

Archivo **.gitignore**:

```
.env
__pycache__/*
*.pyc
```

- Los archivos .env no soportan comentarios en la misma línea que un valor. Esto está MAL:
AZURE_OPENAI_KEY=xxx # mi clave. Python leerá el comentario como parte del valor y la autenticación fallará.

1.6 Errores frecuentes

Error	Causa más probable	Solución
AuthenticationError 401	API Key incorrecta o con comentario en el .env	Copia la KEY con el botón del portal. Elimina cualquier texto tras el valor.
Deployment not found 404	Nombre en AZURE_OPENAI_DEPLOYMENT no coincide	Verifica el nombre exacto en AI Foundry → Model deployments.
Insufficient quota	Cuota agotada en Standard	Cambia el deployment type a Global Standard.
ModuleNotFoundError: openai	Librería no instalada en el entorno activo	Ejecuta pip install openai en el terminal de VSCode.
policy violation al crear recurso	Política institucional o cuota pendiente de liberar	Espera 48h si borraste un recurso recientemente. Si persiste, contacta con el administrador.

2

Chat Playground

Interfaz visual de Azure AI Foundry · Parámetros · View Code

2.1 Acceder al Playground

Desde tu recurso en portal.azure.com → **Go to Azure AI Foundry portal** → en el menú lateral selecciona **Chat** (dentro de Playgrounds).

2.2 Mapa de la interfaz

Zona	Descripción
Setup (izquierda)	Selector de deployment y campo 'Give the model instructions' — aquí escribes el system prompt.
Chat History (centro)	Área de conversación. Escribe mensajes y ve las respuestas en tiempo real.
Parámetros (engranaje)	Icono  en la esquina superior derecha. Abre el panel con temperature, max tokens, top_p y penalties.
View Code (barra sup.)	Genera el código Python equivalente a tu configuración actual. El puente del Playground al script.
Contador de tokens	Abajo del chat: muestra los tokens consumidos en tiempo real (ej. 127/128000).

2.3 Qué puedes probar desde aquí

- **Temperature:** cambia de 0.0 a 1.5 con el mismo prompt y pulsa varias veces — con 0.0 la respuesta es siempre idéntica, con 1.5 varía cada vez.
- **Max Tokens:** ajústalo a 20 y haz una pregunta larga — la respuesta se cortará a mitad de frase.
- **System prompt:** escribe uno profesional y luego haz una pregunta fuera de scope para ver si la restricción funciona.
- **View Code:** tras configurar parámetros y un system prompt, pulsa View Code para ver el Python equivalente a lo que has hecho.

★ No hay entregable en esta sección. Su objetivo es que te familiarices con la interfaz y veas el efecto de los parámetros antes de usarlos desde código en las secciones siguientes.

3

Llamada Simple

Primera llamada a la API desde Python

3.1 Código base — llamada_simple.py

```
# llamada_simple.py
import os
from dotenv import load_dotenv
from openai import AzureOpenAI
load_dotenv()
client = AzureOpenAI(
    azure_endpoint = os.getenv('AZURE_OPENAI_ENDPOINT'),
    api_key = os.getenv('AZURE_OPENAI_KEY'),
    api_version = os.getenv('AZURE_OPENAI_API_VERSION')
)
response = client.chat.completions.create(
    model = os.getenv('AZURE_OPENAI_DEPLOYMENT'),
    messages = [
        {'role': 'system', 'content': 'Eres un asistente técnico útil.'},
        {'role': 'user', 'content': 'Explica qué es una API en dos frases.'}
    ]
)
print(response.choices[0].message.content)
# Ejecutar: python llamada_simple.py
```

3.2 Los tres roles de la API

```
messages = [
    # system: define el comportamiento base – siempre el primero
    {'role': 'system', 'content': 'Eres un experto en Python.'},
    # user y assistant se alternan para simular la conversación
    {'role': 'user', 'content': '¿Qué es un decorador?'},
    {'role': 'assistant', 'content': 'Un decorador es una función que...'},
    # La última entrada es siempre del usuario (la pregunta actual)
    {'role': 'user', 'content': 'Dame un ejemplo práctico.'},
]
# El modelo NO tiene memoria entre llamadas.
# Para simular conversación, enviamos el historial completo cada vez.
```

■ **ENTREGA E1:** Cambia el mensaje del user por una pregunta tuya. Ejecuta el script y haz una captura del terminal mostrando tu pregunta y la respuesta del modelo.

4

Chatbot con Historial

Conversación con memoria · System prompt propio

Crea el archivo **chatbot.py**:

```
# chatbot.py
import os
from dotenv import load_dotenv
from openai import AzureOpenAI
load_dotenv()
client = AzureOpenAI(
    azure_endpoint = os.getenv('AZURE_OPENAI_ENDPOINT'),
    api_key = os.getenv('AZURE_OPENAI_KEY'),
    api_version = os.getenv('AZURE_OPENAI_API_VERSION')
)
# = CAMBIA ESTE SYSTEM PROMPT – ver sección 4.1
historial = [
    {'role': 'system', 'content': 'Eres un asistente técnico útil.'}
]
print('Chatbot Azure OpenAI – escribe "salir" para terminar')
print('-' * 50)
while True:
    pregunta = input('Tu: ').strip()
    if not pregunta: continue
    if pregunta.lower() == 'salir': break
    historial.append({'role': 'user', 'content': pregunta})
    respuesta = client.chat.completions.create(
        model = os.getenv('AZURE_OPENAI_DEPLOYMENT'),
        messages = historial
    )
    mensaje = respuesta.choices[0].message.content
    historial.append({'role': 'assistant', 'content': mensaje})
    print(f'Asistente: {mensaje}')
    print('-' * 50)
# Ejecutar: python chatbot.py
```

4.1 Cómo construir un system prompt profesional

Sustituye el system prompt genérico del código por uno propio con estos cuatro componentes:

```
system_prompt = (
    # 1. Rol específico con credibilidad
    'Eres un experto en [TU ESPECIALIDAD] con X años de experiencia. '
    # 2. Idioma y tono
    'Respondes SIEMPRE en castellano con tono [técnico / amigable / formal]. '
    # 3. Formato de salida
    'Tus respuestas incluyen: (1) explicación concisa, (2) ejemplo práctico. '
    # 4. Restricción negativa – qué NO debe hacer
    'Si la pregunta no es sobre [TU TEMA], responde: "Solo puedo ayudarte con [TEMA]."'
)
```

■ El historial crece con cada mensaje. En producción se limita el número de turnos recordados o se resume periódicamente para controlar el coste.

■ **ENTREGA E2:** Sustituye el system prompt por uno propio con los 4 componentes. Mantén una conversación de al menos 3 turnos y prueba también una pregunta fuera de scope para verificar que la restricción funciona. Captura el terminal completo con la conversación.

5

Parámetros y Prompt Engineering

Referencia — lee antes de pasar al asistente configurable

★ Esta sección es de referencia. Lee el código y los ejemplos para entender los conceptos antes de aplicarlos en el asistente del punto 6, que es donde se evaluará todo junto. No hay capturas que entregar aquí.

5.1 Los 6 parámetros del modelo

Parámetro	Default	Rango	Cuándo usarlo
temperature	1.0	0.0 – 2.0	0.0 extracción/clasificación · 0.7 chatbot · 1.0+ creativo
top_p	1.0	0.0 – 1.0	Ajusta uno u otro, nunca los dos a la vez
max_tokens	4096	1 – 16384	200-500 respuestas cortas · 2000-4000 documentos
frequency_penalty	0.0	-2.0 – 2.0	0.3-0.5 para reducir repeticiones en textos largos
presence_penalty	0.0	-2.0 – 2.0	0.5-0.6 con temperature=0.8 para brainstorming
seed	None	cualquier int	seed=42 durante desarrollo para comparar prompts de forma objetiva

5.2 Experimento: el efecto de temperature

```
# experimento_temperature.py
import os
from dotenv import load_dotenv
from openai import AzureOpenAI
load_dotenv()
client = AzureOpenAI(
    azure_endpoint=os.getenv('AZURE_OPENAI_ENDPOINT'),
    api_key=os.getenv('AZURE_OPENAI_KEY'),
    api_version=os.getenv('AZURE_OPENAI_API_VERSION')
)
deployment = os.getenv('AZURE_OPENAI_DEPLOYMENT')
prompt = "Continúa esta historia en una frase: 'El científico abrió la caja y dentro encontró'"
for temp in [0.0, 0.5, 1.0, 1.5]:
    response = client.chat.completions.create(
        model = deployment,
        messages = [{'role': 'user', 'content': prompt}],
        temperature = temp,
        max_tokens = 60,
        seed = 42
    )
    print(f'temperature={temp:.1f} → {response.choices[0].message.content.strip()}')
```

■ Con temperature=0.0 la respuesta es siempre idéntica (el modelo elige el token más probable). Con 1.5 cada ejecución puede ir en una dirección completamente distinta.

5.3 Técnica 1 — System Prompt profesional

Ya lo has aplicado en la sección 4. Recuerda los 4 componentes: **rol específico** · **idioma/tono** · **formato de salida** · **restricción negativa**.

5.4 Técnica 2 — Few-Shot Prompting

En lugar de explicar el formato, se muestran ejemplos de entrada-salida. Especialmente efectivo para clasificación y extracción de datos.

```
# few_shot.py
ejemplos = [
    ('El producto llegó perfecto y rápido', 'POSITIVO'),
    ('Regular, ni bien ni mal', 'NEUTRO'),
    ('Una basura, se rompió al día siguiente', 'NEGATIVO'),
]

def clasificar_sentimiento(resena):
    shots = '\n'.join(f'Reseña: "{r}" -> {s}' for r, s in ejemplos)
    prompt = f'{shots}\n\nAhora clasifica:\nReseña: "{resena}"'
    resp = client.chat.completions.create(
        model = deployment,
        messages = [
            {'role': 'system', 'content': 'Responde SOLO con: POSITIVO, NEGATIVO o NEUTRO'},
            {'role': 'user', 'content': prompt}
        ],
        temperature=0, max_tokens=10
    )
    return resp.choices[0].message.content.strip()

resenas = [
    'No está mal para el precio que tiene',
    'Increíble, superó todas mis expectativas',
    'Lo peor que he comprado en mi vida',
]

for r in resenas:
    print(f'{clasificar_sentimiento(r):10} | {r}')
```

5.5 Técnica 3 — Chain-of-Thought (CoT)

Pedir al modelo que razone paso a paso antes de dar la respuesta mejora la precisión en tareas lógicas y matemáticas.

```
problema = (
    'Una tienda tiene 3 estantes con 12 libros cada uno. '
    'Vende el 25% el lunes y el 30% de lo restante el martes. '
    '¿Cuántos libros quedan?'
)

# Sin CoT – respuesta directa
r_directo = client.chat.completions.create(
    model=deployment,
    messages=[{'role': 'user', 'content': problema}],
    temperature=0, max_tokens=80
)

# Con CoT – razonamiento explícito
r_cot = client.chat.completions.create(
    model=deployment,
    messages=[
        {'role': 'system', 'content': 'Razona el problema paso a paso antes de dar la respuesta.'},
        {'role': 'user', 'content': problema}
    ],
    temperature=0, max_tokens=200
)

print('=== SIN CoT ==='); print(r_directo.choices[0].message.content)
print('\n=== CON CoT ==='); print(r_cot.choices[0].message.content)
```

Tabla de referencia — parámetros por caso de uso

Caso de uso	temperature	top_p	freq_penalty	max_tokens
Extracción/clasificación	0.0	0.1	0.0	100-200
Respuesta técnica precisa	0.2	0.5	0.0	500-1000
Chatbot general	0.7	0.9	0.3	400-800
Escritura creativa	1.0	1.0	0.6	500-2000
Brainstorming	1.2	1.0	1.0	300-600


```
# asistente.py – Parte 2: funciones y loop principal
def contar_tokens(messages):
    return sum(len(enc.encode(m['content'])) for m in messages)
def chat(historial, pregunta):
    historial.append({'role': 'user', 'content': pregunta})
    system = [{'role': 'system', 'content': CONFIG['system_prompt']}]
    reciente = historial[-(CONFIG['max_historial'] * 2):]
    response = client.chat.completions.create(
        model = deployment,
        messages = system + reciente,
        temperature = CONFIG['temperature'],
        max_tokens = CONFIG['max_tokens'],
        top_p = CONFIG['top_p'],
        frequency_penalty = CONFIG['frequency_penalty'],
        presence_penalty = CONFIG['presence_penalty'],
    )
    respuesta = response.choices[0].message.content
    historial.append({'role': 'assistant', 'content': respuesta})
    return respuesta, response.usage.prompt_tokens, response.usage.completion_tokens
historial = []
print('Asistente · Comandos: /config /reset /tokens salir')
print('=' * 55)
while True:
    entrada = input('Tu: ').strip()
    if not entrada: continue
    if entrada == 'salir': break
    if entrada == '/config':
        [print(f' {k}: {v}') for k, v in CONFIG.items() if k != 'system_prompt']
        continue
    if entrada == '/reset':
        historial = []; print('[Historial borrado]'); continue
    if entrada == '/tokens':
        msgs = [{'role': 'system', 'content': CONFIG['system_prompt']}] + historial
        print(f'[Tokens en contexto: {contar_tokens(msgs)}]'); continue
    resp, p_tok, c_tok = chat(historial, entrada)
    print(f'Asistente: {resp}')
    print(f'[entrada={p_tok} | respuesta={c_tok} tokens]')
    print('-' * 55)
# Ejecutar: python asistente.py
```

Lo que debes personalizar en CONFIG

- **system_prompt**: cámbialo por un system prompt propio con los 4 componentes (rol, idioma/tono, formato, restricción negativa). No puede ser el genérico de ejemplo.
- **temperature y max_tokens**: elige los valores adecuados para el caso de uso de tu asistente según la tabla de referencia de la sección 5.
- **max_historial**: prueba con 2 para comprobar que el modelo 'olvida' los primeros mensajes de una conversación larga.

■ **ENTREGA E3: Personaliza el bloque CONFIG con tu propio system prompt y parámetros. Realiza una sesión de al menos 5 turnos usando los comandos /tokens, /config y /reset. Incluye también al menos 1 pregunta fuera de scope para demostrar que la restricción negativa funciona. Captura el terminal completo.**

Resumen de entregables — Secciones 1 a 6

Nº	Entregable	Qué debe mostrar la captura
E1	Captura llamada_simple.py	Tu pregunta propia y la respuesta del modelo en el terminal.
E2	Captura chatbot.py	System prompt propio (4 componentes) visible y conversación de 3+ turnos con prueba fuera de scope.
E3	Captura asistente.py	CONFIG personalizado, sesión de 5+ turnos con /tokens, /config y /reset visibles, y 1 pregunta fuera de scope rechazada.
E4	Código fuente	Los archivos .py con el código modificado (no el .env).

✓ Los entregables E1 y E2 son independientes: la llamada simple demuestra que el entorno funciona, el chatbot demuestra el system prompt, y el asistente integra parámetros y técnicas de prompting. No son redundantes.